HACKTIV8

Mini Project 6

LLM Monitoring Dashboard

Azure-Free Edition

RAG, evaluation, safety, observability, cost, and demo

NO AZURE REQUIRED

About This Session

Why we are here today

You have learned how to build a basic LLM application. In this mini project, you will turn that knowledge into a measurable product: a RAG assistant that is evaluated, monitored, and defended with data.

4 hr TOTAL DURATION	**4** HOUR-BLOCKS
INDIVIDUAL WORK FORMAT	**1** MINI PROJECT

Core rule: the project must run without Azure. It may use any OpenAI-compatible LLM endpoint, local embeddings, local vector search, and generic deployment.

The Scenario

A real-world problem inside an automotive group

Mitsubishi After-Sales Service Assistant

A customer-facing chatbot for after-sales questions. It is grounded in manuals, warranty terms, parts catalogue, and booking FAQ. It answers in Bahasa Indonesia and English.

RAG-based **Customer-facing** **Bilingual ID/EN**

You build it. You measure it. You defend it.

Why This Matters

Real business pressure creates a real engineering problem

01

Service queries are high-volume and repetitive

Dealers handle many repeated questions every day: service intervals, warranty coverage, booking, and spare parts.

02

Wrong answers cost money and trust

A wrong answer about warranty or service price can damage the brand and create claim risk. Quality and groundedness are not optional.

03

Cost per query decides if it ships

An assistant that is too expensive per question will never survive production traffic. Unit economics matter from day one.

Learning Objectives

By the end of this session you will be able to

1. Stand up a working RAG pipeline using an OpenAI-compatible LLM and local vector search.
2. Apply BLEU, ROUGE, embedding similarity, and groundedness checks to evaluate outputs.
3. Detect and handle toxic, off-topic, and prompt-injection queries without Azure Content Safety.
4. Instrument an LLM application with structured logs and meaningful KPIs.
5. Design and run an A/B experiment that compares prompts, models, or retrieval settings honestly.
6. Justify model tier and architecture against cost, latency, quality, and regulatory constraints.

What You Will Build

System architecture at a glance

DATA LAYER

service_manual.pdf
Local files provided in starter package

warranty_terms.pdf
Local files provided in starter package

parts_catalogue.json
Local files provided in starter package

booking_faq.csv
Local files provided in starter package

The 4-hour Journey

How the session is paced

Setup & Build Baseline

Get a working end-to-end pipeline.

Instrument & Evaluate

Make the system measurable.

A/B Experiment

Change one thing, measure honestly.

Cost Report & Demo

Defend your decisions.

Each hour has a clear deliverable. Falling behind in one hour compounds, so keep moving.

Hour 1

Setup & Build Baseline

A weak baseline you can measure is worth more than a clever one you cannot.

Setup	Create virtualenv, install dependencies, copy .env.example, choose LLM provider, and run FastAPI locally.
Ingest & embed	Load PDFs/JSON/CSV, split into chunks, embed text, and store vectors in ChromaDB or FAISS.
Retrieval + prompt	Retrieve top-k chunks, write a system prompt that constrains the model to answer using retrieved context.
Write the /chat endpoint	Accept query + session_id. Return answer, sources, refusal flag, latency, and status. Smoke-test 5 questions before moving on.

Hour 1 Deliverable

W do

1 Input

User can send a question in ID or EN through the /chat endpoint.

2 Retrieval

System returns relevant chunks and source filenames from local documents.

3 Answer

LLM answers using retrieved context and says "I do not know" when sources are insufficient.

Code expectation: the instructor will provide or create starter code. The slide deck intentionally describes behavior and acceptance criteria, not full implementation snippets.

CHECKPOINT: /health returns OK and /chat returns answer + sources

Hour 2

Instrument & Evaluate

An unmeasured system cannot be defended.

Evaluate quality

- Run BLEU / ROUGE / METEOR on the 30-question gold set.
- Compute embedding similarity between generated answer and expected answer.
- Check groundedness: does the answer only use retrieved sources?

Instrument the system

- Add structured JSON logs for request_id, latency, model, token estimate, and status.
- Log retrieval hit-rate and number of chunks used.
- Create a simple dashboard or report with cost per query, p95 latency, and refusal rate.

Monitoring Metrics

What your dashboard or report should show

p50 / p95 latency

How fast normal and slow requests are.

Token estimate

Input + output token usage per request.

Cost per query

Estimated model + embedding + storage cost.

Retrieval hit-rate

Whether the right source appears in retrieved chunks.

Groundedness rate

Whether answers stay within retrieved evidence.

Refusal rate

How often unsafe or off-topic queries are declined.

For this mini project, a simple CSV report, terminal summary, or screenshot is enough. The goal is measurement discipline, not dashboard polish.

Hour 3

A/B Experiment

Compare one disciplined change against your baseline. Decide with data, not vibes.

VARIANT A: Your baseline

Baseline prompt, current model, current top-k, current safety rule. Already running and already measured.

VS

VARIANT B: Your hypothesis

Change ONE thing only: prompt, model, top-k, chunk size, safety threshold, or reranking. Same gold set and same logging.

Report mean + standard deviation for each metric. Do not move the goalpost after seeing the result.

Hour 4

Cost Report & Demo

Present your decisions. You get 5 minutes: show the dashboard, the numbers, and the trade-offs.

$

Unit cost

Cost per query and daily cost at 10k queries.

↗

Latency

p50 and p95 end-to-end response time.

⚖

Quality + Safety

Eval scores, groundedness, and unsafe-input handling.

▯

Scaling plan

What changes at 100k/day and where it breaks.

Constraint 1

No Azure dependency, but still production-minded

This is not just "replace Azure names". The project should preserve the same engineering concerns: data location, provider portability, cost control, and operational visibility.

DEFAULT - USE THIS

Run locally or on a generic cloud host. Store project data in local files, SQLite, or object storage. Use ChromaDB/FAISS for vector search. Use generic environment variables.

FALLBACK - WHEN NEEDED

Use any OpenAI-compatible API endpoint, local vLLM, Ollama, or managed LLM provider. If data leaves the target region, document why and what data is sent.

Constraint 2

What can it cost per query?

TARGET

≤ IDR 250 per query at 10,000 queries/day. Show your math. We will challenge it.

WHAT IS IN YOUR COST

Chat tokens	*model rate x input/output tokens*
Embedding tokens	*per query + per chunk*
Vector search	*local cost or service tier*
Safety checks	*rule-based or classifier call*
Logging + storage	*logs, reports, dashboard files*

Constraint 3

What happens when users are hostile?

Your gold set will contain at least 5 adversarial queries: toxic language, prompt injection, off-topic asks, and requests to leak internal instructions. Your system must not leak the system prompt or produce unsafe content.

UNACCEPTABLE

Answers off-topic questions Repeats toxic phrasing back to the user Reveals system prompt or model name Invents warranty terms not in corpus Claims certainty when sources are missing

REQUIRED

Politely declines and offers in-scope topics Returns a neutral, brand-safe refusal message Treats system instructions as confidential Says "I do not have that information" when unsure Logs refusal reason without storing sensitive user content

Your Toolbox

Azure-free services you can use

LLM Provider

OpenAI-compatible API, vLLM endpoint, Ollama, or other provider

Vector Store

ChromaDB, FAISS, Qdrant local, or SQLite-based prototype

Safety Layer

Rules, regex, keyword lists, classifier prompt, or guardrails library

Observability

JSON logs, CSV reports, Docker logs, Render/Railway logs, Grafana optional

Storage

Local /data folder, SQLite, S3-compatible storage, or mounted volume

Deployment

Local Docker, Render, Railway, Fly.io, VPS, or internal server

The deck should not force one vendor. The code examples can choose one default stack later.

What is in the Box

Materials we hand you at the start

01 **service_manual.pdf**

PDF - 8 pages
Routine service, fluids, inspection schedules.

02 **warranty_terms.pdf**

PDF - 2 pages
Coverage rules, exclusions, validity periods in ID and EN.

03 **parts_catalogue.json**

JSON - 42 SKUs Part numbers, names, applicable models, indicative prices.

04 **gold_questions.csv**

CSV - 30 + 5 Reference Q&A; pairs plus adversarial queries for safety eval.

All files are anonymized and synthesized for training. Nothing is production data.

Code Examples Later

What the deck should prepare participants to implement

TODO 1

Provider abstraction

A generator interface that can swap OpenAI-compatible API, local vLLM, or fake generator for tests.

TODO 2

Retriever module

Chunking, embedding, vector store creation, search, and source return format.

TODO 3

Safety module

Reject off-topic, toxic, or prompt-injection input before LLM call. Add safe refusal response.

TODO 4

Evaluation runner

Read gold_questions.csv, call /chat, compute metrics, and export eval_report.csv/pdf.

TODO 5

Logging middleware

Structured JSON logs with request_id, latency, token estimate, retrieval stats, and refusal reason.

How You Are Scored

Evaluation rubric: 100 points total

CRITERION	WHAT WE LOOK FOR	WEIGHT
Working system	End-to-end pipeline. /chat returns grounded answers under p95 target.	**25**
Evaluation quality	BLEU / ROUGE / embedding similarity / groundedness with method described.	**20**
Safety handling	Adversarial gold set blocked or refused cleanly. No prompt leakage.	**15**
A/B experiment	Single-variable change, written hypothesis, honest result with std-dev.	**15**
Cost discipline	Unit cost calculated, target explained, scaling plan to 100k/day.	**15**
Demo + defense	Clear 5-minute demo. Honest about weak spots. Strong answers to Q&A.;	**10**

What You Submit

Submission package - exact contents

One ZIP file, named mini-project-6-{your-name}.zip

src/

Source code folder. Full repo with README and run steps.

eval_report.pdf

3-5 pages. All metrics, method, gold set result, and A/B comparison.

dashboard.png

Screenshot or generated report showing tokens, p95 latency, refusal rate, unit cost.

architecture.png

Single-page diagram showing app, data, retrieval, safety, evaluation, and logs.

cost_analysis.pdf

1 page. Per-query IDR cost, monthly estimate, and scaling plan.

How You Work

This is an individual project; you own every line

YOU WEAR FOUR HATS

B	**Builder**	Own the RAG pipeline and the /chat endpoint.
E	**Evaluator**	Own the gold set, metrics, and dashboard.
O	**Optimizer**	Own the A/B variant, cost math, and report.
P	**Presenter**	Own the demo storyline and 5-minute pitch.

GROUND RULES

Commit code often Log every failure Ask for help if stuck > 10 min Document external help in README Be honest about what did not work

Hard Won Advice

Things participants wish they had known

✓ DO

Get a weak baseline in Hour 1, then improve. Build the evaluation harness before optimizing the prompt. Log token counts from the first request. Cache embeddings; never re-embed corpus on every demo. Test 5 questions manually before trusting any metric.

✕ DON'T

Spend an hour picking the perfect model. Optimize one metric while ignoring the others. Hide failures in the demo; they will come up in Q&A.; Hardcode answers as fallbacks. We will check. Skip the cost slide. It is the whole point.

HACKTIV8

Thank You

Build it. Measure it. Defend it - without Azure dependency.